Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2025-26

Class:	**TE**	**Semester:**	**V**
Course Code:	**CSC502**	**Course Name:**	**WC**

Name of Student:	**Pranita Kumbhar**
Roll No. :	**70**
Experiment No.:	**06**
Title of the Experiment:	**Node.Js: Installation and Configuration, Callbacks, Event loops, Creating express**
Date of Performance:	**03/09/25**
Date of Submission:	**10/09/25**

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : Ms. Kshitija Gharat

Signature :

Date:

Aim: To implement file system operations and path handling in Node.js, and to design a simple REST API using Express framework with middleware functions.

Objective:
To understand the usage of File System (fs) module for reading, writing, and updating files
To learn how to work with path module for handling file and directory paths
To develop RESTful API endpoints using Express framework
To apply middleware in Express for logging and request handling

Requirement:
Node.js installed (version 18 or above recommended)
Express framework (installed via npm)
Text editor such as VS Code
Web browser or Postman for API testing
Theory:
Node.js provides inbuilt modules and frameworks for backend development.
File System Module (fs):
This module allows reading, writing, updating, and deleting files. Example functions include fs.readFileSync and fs.writeFileSync.
Path Module (path):
This module helps in working with file and directory paths. Example usage is path.join(__dirname, 'file.txt').
Express Framework:
Express is a lightweight Node.js framework to build web servers and REST APIs. It provides routing, middleware, and response handling.
Middleware in Express:
Middleware functions are executed between request and response. They are used for logging, authentication, validation, and other purposes.

Procedure:
Step 1: Install Node.js and initialize the project using the following commands
```
mkdir node-experiment
cd node-experiment
npm init -y
npm install express
```
Step 2: Create server.js file and include required modules
```
const fs = require('fs');
const path = require('path');
const express = require('express');
const app = express();
const PORT = 3000;
```
Step 3: Perform File System operations
```
fs.writeFileSync('sample.txt', 'Hello, Node.js FS Module!');
```

```
const data = fs.readFileSync('sample.txt', 'utf8');
console.log("File Content:", data);
```

Step 4: Use Path Module

```
const filePath = path.join(__dirname, 'sample.txt');
console.log("Absolute File Path:", filePath);
```

Step 5: Create Express REST API with Middleware

```
app.use((req, res, next) => {
console.log(${req.method} ${req.url});
next();
});
app.get('/', (req, res) => res.send('Welcome to Node.js REST API'));
app.get('/data', (req, res) => res.json({ message: "Hello World" }));
app.listen(PORT, () => {
console.log(Server running at http://localhost:${PORT});
});
```

Step 6: Run the server using the command

```
node server.js
```

Step 7: Test endpoints using browser or Postman

http://localhost:3000/ → Displays welcome message

http://localhost:3000/data → Returns JSON response

Code-

```
const fs = require('fs');
const path = require('path');
const express = require('express');

const app = express();
const PORT = 3000;

// Middleware to parse JSON
app.use(express.json());

// ==============================================
// MIDDLEWARE: Logging Middleware
// ==============================================
app.use((req, res, next) => {
   const timestamp = new Date().toLocaleTimeString();
   console.log(`[${timestamp}] ${req.method} ${req.url}`);
   next();
});
```

```
// ==============================================
// FILE SYSTEM OPERATIONS (fs module)
// ==============================================

// Create and write to file
const fileName = 'sample.txt';
const fileContent = 'Hello, Node.js FS Module! This is sample data.';

fs.writeFileSync(fileName, fileContent);
console.log('✓ File created and written successfully');

// Read file synchronously
const data = fs.readFileSync(fileName, 'utf8');
console.log('✓ File Content:', data);

// ==============================================
// PATH MODULE
// ==============================================

const filePath = path.join(__dirname, fileName);
console.log('✓ Absolute File Path:', filePath);

// ==============================================
// REST API ENDPOINTS
// ==============================================

// 1. Home Route
app.get('/', (req, res) => {
  res.send(`
    <h1>Welcome to Node.js REST API</h1>
    <p>Available Endpoints:</p>
    <ul>
      <li>GET / → Welcome message</li>
      <li>GET /data → JSON response</li>
      <li>GET /file → File system data</li>
      <li>GET /path → Path module info</li>
      <li>GET /users → Get all users</li>
      <li>POST /users → Add new user</li>
      <li>GET /users/:id → Get user by ID</li>
    </ul>
  `);
});
```

```
// 2. JSON Data Endpoint
app.get('/data', (req, res) => {
    res.json({
        message: "Hello World",
        status: "success",
        timestamp: new Date()
    });
});

// 3. File System Endpoint
app.get('/file', (req, res) => {
    res.json({
        message: "File System Module Demo",
        fileName: fileName,
        filePath: filePath,
        content: data,
        fileExists: fs.existsSync(fileName)
    });
});

// 4. Path Module Endpoint
app.get('/path', (req, res) => {
    res.json({
        message: "Path Module Demo",
        absolutePath: filePath,
        fileName: path.basename(filePath),
        directory: path.dirname(filePath),
        extension: path.extname(filePath),
        parseInfo: path.parse(filePath)
    });
});

// ==============================================
// SAMPLE DATABASE (Users Array)
// ==============================================
let users = [
    { id: 1, name: "Kavya Naik", email: "kavya@example.com" },
    { id: 2, name: "Pranita Kumbhar", email: "pranita@example.com" },
    { id: 3, name: "John Doe", email: "john@example.com" }
];

// 5. GET all users
app.get('/users', (req, res) => {
```

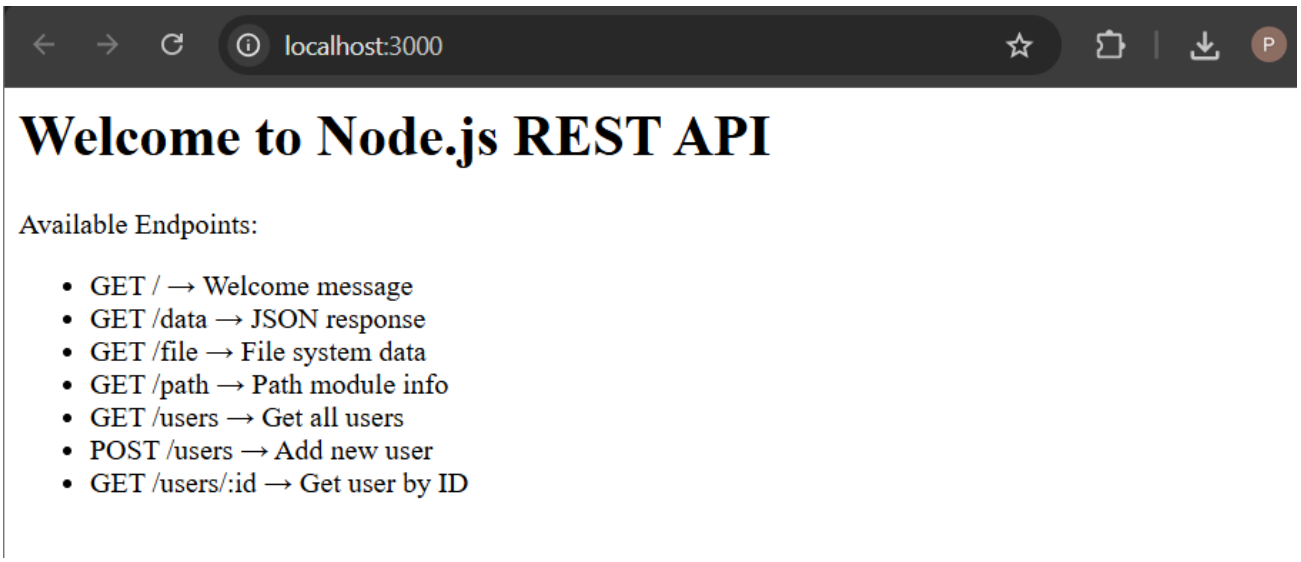
```
    res.json({
        message: "All Users",
        count: users.length,
        data: users
    });
});

// 6. GET user by ID
app.get('/users/:id', (req, res) => {
    const userId = parseInt(req.params.id);
    const user = users.find(u => u.id === userId);

    if (user) {
        res.json({
            message: "User Found",
            data: user
        });
    } else {
        res.status(404).json({
            message: "User Not Found",
            id: userId
        });
    }
});

// 7. POST - Create new user
app.post('/users', (req, res) => {
    const newUser = {
        id: users.length + 1,
        name: req.body.name || "Unknown",
        email: req.body.email || "unknown@example.com"
    };

    users.push(newUser);

    res.status(201).json({
        message: "User Created Successfully",
        data: newUser
    });
});

// 8. PUT - Update user
app.put('/users/:id', (req, res) => {
```

```
    const userId = parseInt(req.params.id);
    const user = users.find(u => u.id === userId);

    if (user) {
        user.name = req.body.name || user.name;
        user.email = req.body.email || user.email;
        res.json({
            message: "User Updated Successfully",
            data: user
        });
    } else {
        res.status(404).json({
            message: "User Not Found"
        });
    }
});

// 9. DELETE - Remove user
app.delete('/users/:id', (req, res) => {
    const userId = parseInt(req.params.id);
    const index = users.findIndex(u => u.id === userId);

    if (index !== -1) {
        const deletedUser = users.splice(index, 1);
        res.json({
            message: "User Deleted Successfully",
            data: deletedUser[0]
        });
    } else {
        res.status(404).json({
            message: "User Not Found"
        });
    }
});

// 10. Error Handling Middleware
app.use((req, res) => {
    res.status(404).json({
        message: "Endpoint Not Found",
        path: req.url,
        availableEndpoints: [
            "GET /",
            "GET /data",
```

```
        "GET /file",
        "GET /path",
        "GET /users",
        "GET /users/:id",
        "POST /users",
        "PUT /users/:id",
        "DELETE /users/:id"
      ]
    });
});

// ==============================================
// START SERVER
// ==============================================
app.listen(PORT, () => {
    console.log('\n' + '='.repeat(50));
    console.log(' Server running at http://localhost:' + PORT);
    console.log('='.repeat(50) + '\n');
});
```

Output:

```
✓ File created and written successfully
✓ File Content: Hello, Node.js FS Module! This is sample data.
✓ Absolute File Path: C:\Users\Pranita Kumbhar\Downloads\node-experiment\node-experiment\sample.txt

==================================================
🚀 Server running at http://localhost:3000
==================================================

[11:37:06 pm] GET /
```



Conclusion:
Node.js provides powerful modules like fs and path for file handling. Using Express, developers can easily create REST APIs and enhance functionality with middleware. This experiment demonstrates integration of core Node.js modules with Express framework for backend development.